Computer Architectures 02LSEOV **Laboratory 6**	Delivery date: November 22th 2025, 11.59 PM Expected delivery of lab_06.zip must include: - Solutions of exercises 1, 3, and 4. - This file, filled with information and possibly compiled in a pdf format.

This lab will continue to delve into ARM programming.

1) Write a program using the ARM assembly that performs the following operations:
   1. Initialize registers *R3*, *R4*, and *R5* with arbitrary (chosen by you) signed values.
   2. Subtract *R3* to *R4* (*R4 – R3*) and store the result in *R6*.
   3. Sum *R4* to *R5* (*R4 + R5*) and store the result in *R7*.
2) Using the debug log window, change the values of the written program in order to set the following flags to 1, one at a time and when possible:
   - carry
   - overflow
   - negative
   - zero

   Report the selected values in the table below:

Updated flag	Hexadecimal representation of the obtained values			
	R4 – R3		R4 + R5	
	R4	R3	R4	R5
Carry = 1	0x0000000A	0x00000005	0xFFFFFFFF	0x00000002
Carry = 0	0x00000002	0x00000001	0x00000001	0x00000001
Overflow	0x80000000	0x00000001	0x7FFFFFFF	0x00000001
Negative	0xFFFFFFFE	0xFFFFFFFF	0xFFFFFFFF	0x00000000
Zero	0x00000001	0x00000001	0x00000000	0x00000000

Please explain the cases where it is **not** possible to force a **single** FLAG condition:
- Non è possible ottenere il bit di overflow V=1 senza che il flag negativo N sia 1 poiché l'overflow va a sovrascrivere con 1 il bit di segno trasformando il numeo in negativo, in tuttui gli altri casi viene imposta il bit carry C a 1

3) Write two versions of a program that performs the following operations:
   1. Initialize registers *R1* and *R2* with arbitrary (chosen by you) signed values.
   2. Compare the two registers:
      - If they differ, store in register *R8* the maximum among *R1* and *R2*.
      - Otherwise, perform a logical right shift of 1 on *R1* (is it equivalent to what?), then subtract this value from *R2* and store the result in *R9* (i.e., *R4 = R2 – (R1 >> 1)*).

   The first version should be based on a traditional assembly programming approach using conditional branches, while the second version should be based on a conditional statement execution approach.
   Considering a CPU clock frequency (clk) of *16 MHz*, report the number of clock cycles (cc) and the simulation time in milliseconds (ms) in the following table:

	R2 == R3 [cc]	R2 == R3 [ms]	R2 != R3 [cc]	R2 != R3 [ms]
1) Traditional	10,72	0,00067	14,72	0,00092
2) Conditional Execution	13,28	0,00083	13,28	0,00083

Note: you can change the CPU clock frequency by following the brief guide at the end of the document.

4) Write a program that calculates the parity bit of a variable. The parity bit is used for error detection and indicates whether the number of 1s in the binary data is odd (parity = 1) or even (parity = 0), calculated by performing an XOR operation on all bits of the variable: for example, the parity of 0b00000101 is 0 (even parity) because it has two 1s. The variable to be checked is in R10 that you have to initialize to 0b00110101. After calculating the parity, store the result (0 or 1) in R13.

   To recap, implement ASM code that does the following:
   - Calculate the parity bit of R10 by XORing all its bits together.
   - Store the parity bit result in R13 (0 for even parity, 1 for odd parity).

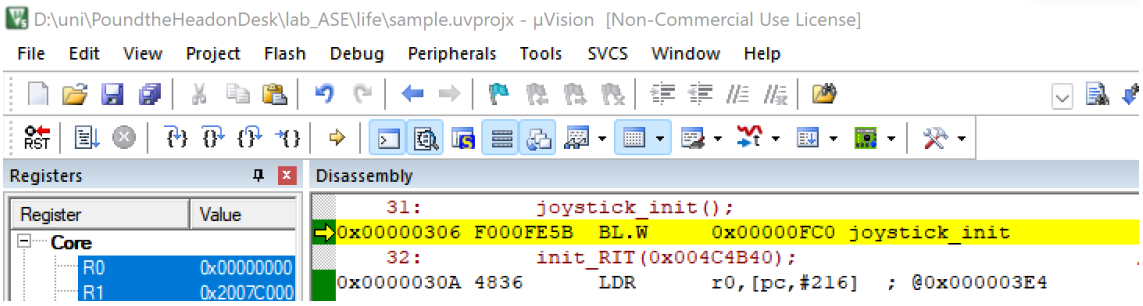
   a) Assuming a *15 MHz* clk, report the code size and execution time in the following table:

Code size [Bytes]	Execution time
564	0,00000625 s

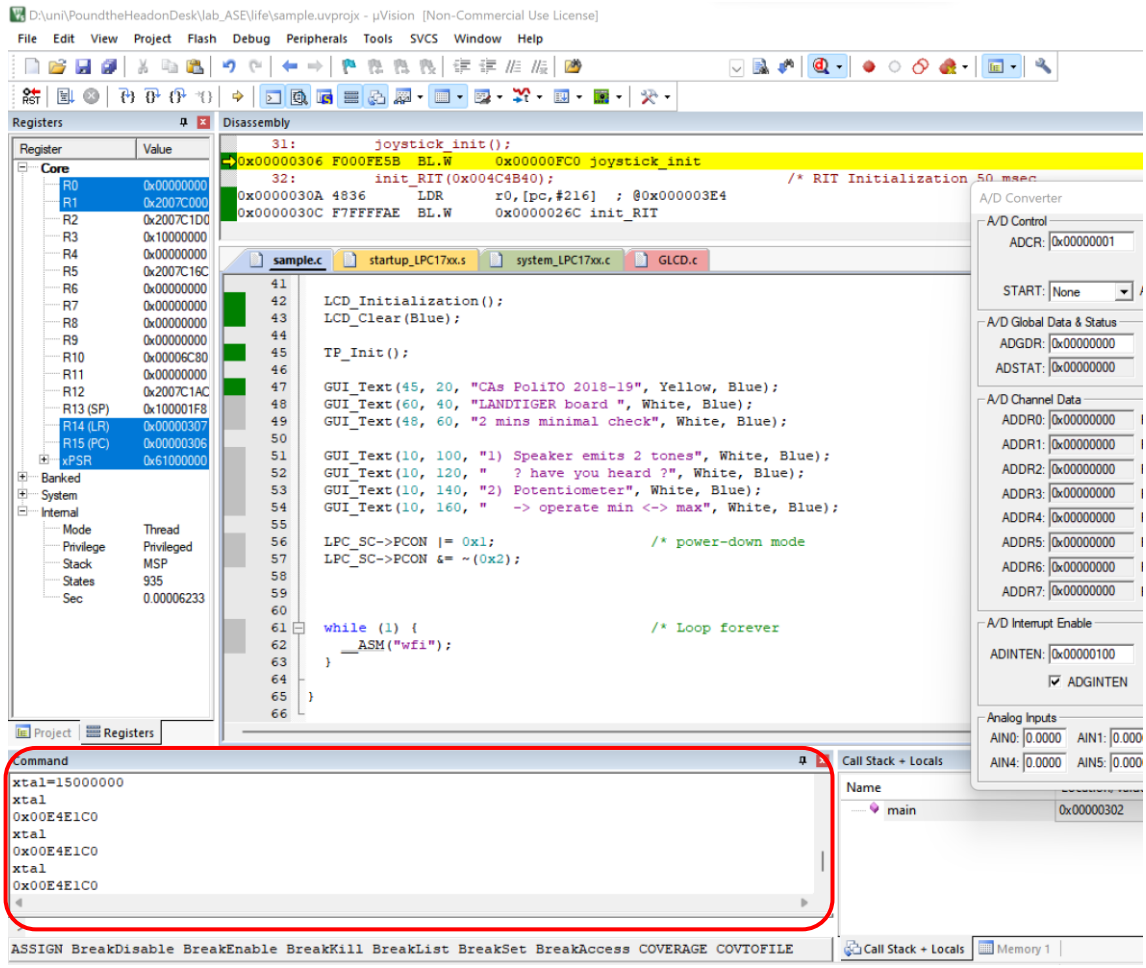
   b) Add some comments regarding the solution you wrote: Ad ogni iterazione applico una maschera con un operazione di AND contenente un solo uno che poi viene shiftata a sinistra per prendere in considerazione il bit successivo. Se l'applicazione della maschera è diverso da zero incremento il contatore di "1". Al termine del loop applico una maschera al contatore che contiene un solo uno al LSB ottenenedo così la parità.

How to set the CPU clock frequency in Keil

1) Launch the debug mode and activate the command console.



2) A window will appear:



You can type *xtal* to check its value. To change its value, make a routine assignment, i.e., *xtal=frequency*, keeping in mind that frequency is in Hz must be entered. To set a frequency of *15 MHz*, you must write as follows: *xtal=15000000*.